Z7i Assignment App

Software Architecture · Handover document · 21 October 2026

1. Overview

Z7i Assignment App (launcher label "Zeal") is a Flutter app in which students pick a subject, browse topic-wise assignments, download a question paper, attempt it offline against a 30-minute timer, review the result and upload it to the backend. It uses GetX for state and navigation, get_it for service lookup, GetStorage as an offline cache, and a Dio-based `ApiService` that talks to the IMS backend.

Area	Choice
Framework	Flutter, Dart SDK ^3.10.1
App version	1.0.1+1 (`pubspec.yaml`)
Platform	Android; layouts target tablets (shortest side >= 600 dp) with a phone fallback
State management	GetX (`get` ^4.6.6): controllers, Rx observables, Obx and GetBuilder
Navigation	GetX named routes (`GetMaterialApp` + `GetPage`). go_router ^14.2.7 is declared but not used for routing
Dependency lookup	get_it ^7.7.0 via `ServiceLocator`; GetX `Get.put / Get.find` for Session and StorageService
HTTP	dio ^5.4.0 for all calls
Local storage / offline	get_storage ^2.1.1 (cache of subjects, topics, papers, results); connectivity_plus ^5.0.2
Maths rendering	flutter_tex ^5.1.10 (TeXView for question HTML); flutter_math_fork and latext are declared but effectively unused
Device management	managed_configurations ^1.0.2 (MDM silent login)
Web content	webview_flutter ^4.13.1 (Usage screen, local assets/usage.html)

2. Getting started

You need a Flutter stable SDK that ships Dart 3.10 or newer, and Android Studio with JDK 17 and the Android SDK.

1. `flutter pub get`
2. Run on a tablet or emulator: `flutter run`
3. Release build: `flutter build apk --release`

Setting	Where	Value today
API base URL	`lib/core/constants/app_constants.dart` (`AppConstants.baseUrl`)	`https://portalapi.z7i.in/api/crm`
Timeout	`AppConstants.timeout`	30 s (connect and receive)
Android application ID	`android/app/build.gradle.kts`	`com.suguna.suguna_assignment_app`
Android release signing	`android/key.properties` + `android/app/key.jks` (both git-ignored)	Ask the current owner for these files

There are no build flavours or environment files. Other backends (IP:3012 servers) are listed as commented-out `baseUrl` lines in `AppConstants`; switch by editing that file.

The Gradle script reads `key.properties` without a fallback (`keystoreProperties["storeFile"] as String`), so any Android build fails with a cast error if that file is missing. Create it (or restore the debug signing line) before your first build.

3. Project structure

All app code (66 Dart files) lives in `lib/`. Shared code is in `lib/core/`, models and storage in `lib/data/`, and each screen family is a feature folder under `lib/modules/`.

Path	What it holds
`lib/main.dart`	Startup, MDM silent login, `MyApp` (`GetMaterialApp`)
`lib/core/routes/`	`app_routes.dart` (path constants), `route_config.dart` (the GetPage list)
`lib/core/services/`	`api_service.dart` (Dio base), `assigment_api_services.dart` (every endpoint), `service_locator.dart`
`lib/core/constants/`	`AppConstants`: base URL, timeout, layout sizes
`lib/core/widgets/`	AppLayout + AppSidebar + AppHeader (shell), AppKeyboard (on-screen digit keypad), AppText; AppDrawer, AppAppBar, AppRichText are unused
`lib/core/theme/, utils/, resources/`	`AppColors`, `AppTheme` (light only), `ResponsiveHelper`, `AppStrings`
`lib/data/models/`	SubjectModel, TopicModel / SubtopicModel / AssignmentScore, Bookmark, QuestionModel (mock only)
`lib/data/services/`	`StorageService` (GetStorage wrapper), `Session` (in-memory state), `NetworkService`, `MockDataService`
`lib/modules/<feature>/`	One folder per feature (see section 6)
`assets/`	`test.json` (sample paper, still loaded by the parser), `usage.html` (Usage screen)

Most feature folders use the GetX three-part layout:

- **bindings/**: a Bindings class that `Get.lazyPuts` the controller. It is attached to the GetPage, so GetX runs it before building the view.
- **controllers/**: a GetxController that loads data in `onInit()` and exposes .obs fields or calls `update()`.
- **views/**: widgets that rebuild through Obx or `GetBuilder<TestPlayerController>`. Several views branch on `ResponsiveHelper.isTablet(context)`.

Exceptions: `test_player/` holds the controller, parser and the player, review and options views shared by three routes; `file_manager/` does its API work inside the view; `test_analysis/` has a controller that is never registered (its binding registers TestPlayerController).

4. Architecture & patterns

Every screen follows one path: the GetPage runs its binding, the binding creates the controller, the controller checks connectivity and calls AssignmentApiServices, which sends the request through the shared Dio client. Responses are cached in GetStorage so the app works offline; the student IDs come from the saved login response, and the current selection (subject, class, test) is held in the in-memory Session.

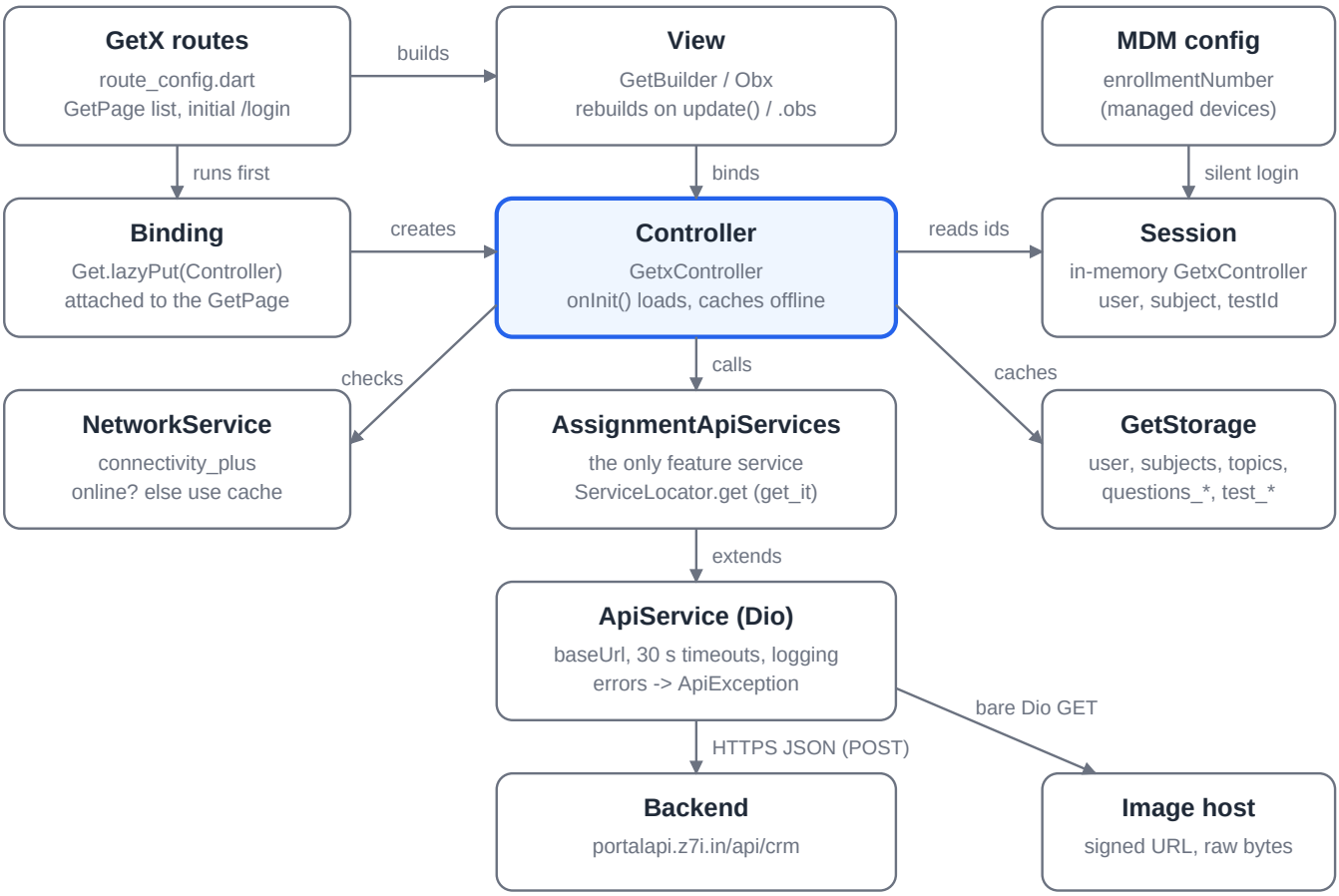

Figure 1. Request path for a typical screen, with routing, connectivity check, session, offline cache and MDM login around it.

Startup sequence (lib/main.dart)

1. `WidgetsFlutterBinding.ensureInitialized()`
2. Start the TeX rendering server (`TeXRenderingServer.start()`).
3. `Get.put(StorageService(), permanent: true)` and `Get.put(Session())`.
4. `GetStorage.init()`: opens the local store.
5. `ServiceLocator.setupServices()`: registers ApiService and AssignmentApiServices in get_it.
6. MDM silent login: if the device policy supplies enrollmentNumber and no "user" is stored, call /student/login with it and save the response.
7. `runApp(MyApp())`: `GetMaterialApp` with `initialRoute: /login`. LoginController.onReady() sends a stored user straight to /home.

State management (GetX)

Controllers hold state as Rx fields (`isLoading = false.obs`, `subjects = <SubjectModel>[].obs`) or plain fields refreshed with `update(['palette'])`. `TestPlayerController` is the hub of the test flow: it is lazily registered with `fenix: true` by the player and analysis bindings, reached through `TestPlayerController.to`, and shared by the player, analysis and review screens. `Session` is a GetxController used as a global bag of the current user, subject, topic, class, test id and flags (isNewTest, isRetest, isCompletedAssignment).

Navigation (GetX routes)

`RouteConfig.getPages()` lists 12 GetPages: /login, /home, /topic-assignments, /assignment-instructions, /test-player, /test-analysis, /test-review, /question-review, /usage, /file-manager, /discussion, /reset. There is no route guard; the login check lives in `LoginController.checkLogin()`. Data is passed with `Get.arguments` (for example the SubjectModel to /topic-assignments) and through Session. Some screens are pushed without a route (`Get.off(() => TestAnalysisView())`). The sidebar offers Home, Usage, File Manager and Reset.

Dependency lookup (get_it)

`ServiceLocator` wraps GetIt and registers `ApiService` and `AssignmentApiServices` as lazy singletons; controllers fetch the latter with `ServiceLocator.get<AssignmentApiServices>()`. StorageService and Session are GetX singletons. LoginController and TopicAssignmentsController also create their own `StorageService()`; that works because both wrap the same GetStorage box.

Offline cache (GetStorage keys)

Key	Written by	Holds
`user`	Login, MDM login	Full /student/login response; "logged in" means this key exists
`subjects`	Home	Last /student/get/assnsubjects response
`topics`	Topic assignments	Map of `topics_<subjectId>_<classNo>` -> /student/get/assignments response
`questions_<subtopicId>-<category>-<testId>`	Topic assignments	Downloaded paper with images embedded as base64 data URIs
`test_<testId>`	Test player (Submit Test)	Paper, local result and attempt count; removed after a successful upload
`bookmarks_<testId>`	Test player	Bookmarked question ids

5. Networking & services

Every backend call goes through AssignmentApiServices, a subclass of ApiService that wraps one Dio client. Requests carry no auth header: the backend identifies the student by entiId (tenantId) and regNo / admissionNo / studentId sent in each body. All endpoints are POST.

ApiService (base)

- Dio with `AppConstants.baseUrl`, 30 s timeouts, JSON headers and a `LogInterceptor` that logs full request and response bodies.
- Only `get` and `post` exist (get is unused). Both return Map<String, dynamic>; a non-map body is wrapped as `{success: true, data: ...}`.
- Errors become `ApiException(message, statusCode)`, with the message taken from the response's `message` field or "An error occurred".
- `downloadImageBytes(url)` uses a separate bare Dio (no base URL, no JSON headers) to fetch image bytes from signed URLs.
- Endpoints answer `{status: true, data: ...}`; controllers check `response['status']`.

Session and login

- `login(regNo)` posts `{regNo}` to /student/login. There is no password; the login screen collects only an enrolment number.
- On success the response is saved under `user` and copied to `Session.currentUser`. Later calls read `data.tenantId`, `data.admissionNo`, `data.studentId`, `data.batchId`, `data.classNo`, `data.name`, `data.currentAcademicYear` and `data.ayHistory` from it.
- There is no logout: the only Logout item (AppDrawer) is unused and its handler is a TODO. Reset keeps the user.

MDM silent login

On managed devices the MDM pushes `enrollmentNumber` through managed_configurations. main.dart logs in with it before runApp if no user is stored. It only runs at cold start; config changes while the app is running are not picked up.

Service methods and endpoints

AssignmentApiServices method	Endpoint	Used by
login	/student/login	Login, main.dart (MDM)
getAssignments	/student/get/assnsubjects	Home
getAssignmentTopics	/student/get/assignments	Topic assignments
loadQuestions	/student/download/assignments	Topic assignments
getAnswers	/assignment/result/list/student?testId=	Topic assignments
downloadQuestionImage	/assignment/question/image/download	Topic assignments
submitResult	/assignment/result/submit	Test analysis, File manager
loadBookmarks	/assignment/question/bookmark/list/?studentId=	Test player controller
bookMarkQuestion	/assignment/question/bookmark/create	Test review
removeBookmark	/assignment/question/bookmark/remove	Test review
reportQuestion	/assignment/question/report/create	Test player, Test review
ApiService.downloadImageBytes	GET <signed or original image URL>	Topic assignments

Error handling differs per method. `submitResult` catches errors and *returns* the exception, so callers test `response is Exception`. The bookmark and report methods do not catch, so their callers' identical "is Exception" checks never fire and failures surface as uncaught async errors.

6. Feature modules

The flow is linear: Login -> Home (subjects) -> Topic assignments -> Instructions -> Test player -> Test analysis -> Test review. The sidebar adds Usage, File manager and Reset.

Module	Route(s)	What it does	API	Entry
auth	/login	Enrolment number login with the on-screen digit keypad	Yes	Start
home_assignments	/home	Subject list, academic-year picker, current class	Yes	After login
topic_assignments	/topic-assignments	Topic -> subtopic grid of assignment dots by exam category; downloads papers; locks an assignment until the previous one is done	Yes	Tap subject
assignment_instructions	/assignment-instructions	Instructions popup, Start button	No (mock text)	Start assignment

Module	Route(s)	What it does	API	Entry
test_player	/test-player, /test-review	Timed test (30 min), palette, keypad for numericals, report question; review with answers, bookmarks	Yes	Accept instructions
test_analysis	/test-analysis	Score summary, per-question table, Reattempt and Finalize (upload)	Yes (via TestPlayerController)	After Submit Test, or tap a finished dot
file_manager	/file-manager	Lists completed results stored locally and uploads them	Yes	Sidebar
usage	/usage	Shows assets/usage.html in a WebView	No	Sidebar
question_review	/question-review	Question review on mock data	No	Unreachable
discussion_list	/discussion	Placeholder text	No	Unreachable
reset	/reset	Placeholder view; the sidebar Reset button clears storage itself	No	Unreachable

7. Screen-wise API calls

Every call below is a POST relative to `https://portalapi.z7i.in/api/crm` with a JSON body, unless noted. Body values come from the saved login response (GetStorage "user" / Session.currentUser) and the Session:

- `entiId` = user.data.tenantId
- `regNo` / `admissionNo` = user.data.admissionNo (the login request itself sends the typed number as regNo)
- `studentId`, `batchId`, `classNo` (in submissions), `studentName` = user.data.studentId / batchId / classNo / name
- `classNo` (in topic and paper requests) = Session.currentClass, from the ayHistory entry of the selected academic year
- `subjectId` = the subject **name** in topic and paper requests, but the subject **id** in submissions and bookmarks

Errors usually surface as a snackbar with the backend message or a generic text.

Login *(auth/)*

When	Method + endpoint	Request body / params	Service method
Tap Login	POST /student/login	{regNo}: the enrolment number typed in. Status true -> save user, go to /home	AssignmentApiServices .login
App start on an MDM device (before the login screen)	POST /student/login	{regNo}: enrollmentNumber from MDM config	AssignmentApiServices .login
Screen ready (onReady)	none	If "user" is stored, go straight to /home	-

Home (subjects) *(home_assignments/)*

When	Method + endpoint	Request body / params	Service method
Screen opens (onInit) and academic-year change, when online	POST /student/get/assnsubjects	{entiId, regNo}. The year change re-sends the same body; only Session.currentClass changes	AssignmentApiServices .getAssignments
Same triggers, offline	none	Reads cached "subjects"	-
Tap a subject	none	Saves it to Session, opens /topic-assignments	-

Topic Assignments *(topic_assignments/)*

When	Method + endpoint	Request body / params	Service method
Screen opens (onInit), online	POST /student/get/assignments	{entiId, regNo, subjectId: subject name, classNo}. Offline: cached topics_<subjectId>_<classNo>	AssignmentApiServices .getAssignmentTopics
Tap a dot with status ASSIGNED that is not cached yet	POST /student/download/assignments	{entiId, regNo, testId, subjectId: subject name, classNo}. Saved as questions_<key>	AssignmentApiServices .loadQuestions
Right after the call above, once per image (also when a cached paper still has image URLs and the device is online)	POST /assignment/question/image/download	{url}: image URL from question, option, passage, column or solution HTML, or images[].url. Returns a signed url	AssignmentApiServices .downloadQuestionImage
Right after each signed-URL call (up to 2 attempts)	GET <signed url, or the original url> (outside the base URL, raw bytes)	none. Bytes are embedded in the HTML as a base64 data URI	ApiService .downloadImageBytes
Tap a dot whose status is not ASSIGNED (1st call)	POST /student/download/assignments	{entiId, regNo, testId, subjectId: subject name, classNo}	AssignmentApiServices .loadQuestions
Same tap (2nd call), then opens /test-analysis	POST /assignment/result/list/student?testId={testId}	{entiId, admissionNo, batchId, testId}	AssignmentApiServices .getAnswers
Tap Start assignment	none	Opens Test analysis if test_<testId> is complete, else Instructions	-

Test Player *(test_player/views/test_player_view.dart)*

When	Method + endpoint	Request body / params	Service method
Report dialog -> Submit (comment required)	POST /assignment/question/report/create	{entiId, studentId, studentName, questionId, assignmentId: current testId, issueType: always "WRONG_ANSWER", description: comment}	AssignmentApiServices .reportQuestion
Submit Test (type SUBMIT to confirm)	none	Result saved locally under test_<testId>, then Test analysis	-

Test Analysis *(test_analysis/views/test_analysis_view.dart)*

When	Method + endpoint	Request body / params	Service method
Screen opens with a stored result (TestPlayerController.onInit -> initData), not on a retest	POST /assignment/question/bookmark/list/?studentId={studentId}	{entiId}	AssignmentApiServices .loadBookmarks
Tap Finalize -> Submit	POST /assignment/result/submit	{testId, subjectId, classNo, admissionNo, entiId, batchId, startedAt, numberOfReattempts, answers[{questionId, answerRaw}], completedAt} (epoch ms). Success removes test_<testId> and returns to /home	AssignmentApiServices .submitResult
Tap Reattempt	none	Sets Session.isRetest, reopens Instructions	-

Test Review *(test_player/views/test_review_view.dart (route /test-review))*

When	Method + endpoint	Request body / params	Service method
Tap the bookmark icon (not bookmarked)	POST /assignment/question/bookmark/create	{entiId, studentId, questionId, assignmentId: testId, subjectId, note: fixed text "Tricky concept, revise before finals"}	AssignmentApiServices .bookMarkQuestion
Tap the bookmark icon (bookmarked)	POST /assignment/question/bookmark/remove	{entiId, bookmarkId}: from the list loaded above, or "" if not found	AssignmentApiServices .removeBookmark
Report dialog -> Submit	POST /assignment/question/report/create	Same body as in the Test Player	AssignmentApiServices .reportQuestion

File Manager *(file_manager/views/file_manager_view.dart)*

When	Method + endpoint	Request body / params	Service method
Screen opens	none	Lists every stored test_* entry with completed == true	-
Tap Upload -> confirm	POST /assignment/result/submit	{testId, subjectId, classNo, admissionNo, entiId, batchId, startedAt, numberOfReattempts, answers[{questionId, answerRaw}], completedAt}, built from the stored result	AssignmentApiServices .submitResult

No API calls: Assignment Instructions (mock text), Usage (local HTML), Question Review (mock data, and unreachable), Discussion and Reset (placeholders). The sidebar Reset button only erases local storage.

Defined but not called: every AssignmentApiServices method is used. Unused code around it: `ApiService.get`, the get_it registration of plain ApiService, `MockDataService.getSubjects / getTopics`, `TestPlayerController.submitTest`, `TestAnalysisController`, `AssignmentQuestionsController` and `_showSubmitConfirmation` in test_review_view.dart.

8. Platform specifics

The app runs on Android tablets, optionally under MDM, and is designed to work offline once papers are downloaded.

Concern	How it works	Where
Tablet vs phone	ResponsiveHelper.isTablet() = shortest side >= 600 dp; several views build separate tablet and mobile layouts. Orientation is not locked	core/utils/responsive_helper.dart
Offline	NetworkService.hasInternet() (connectivity_plus) decides between API and GetStorage cache; images are embedded as base64 so papers render offline	data/services/, topic_assignments_controller.dart
MDM config	managed_configurations reads enrollmentNumber at cold start	main.dart
Maths rendering	TeXRenderingServer.start() at startup; flutter_tex TeXView renders question HTML	main.dart, question_options_widget.dart
On-screen keypad	AppKeyboard (digits layout) for the login number and numerical answers	core/widgets/app_keyboard.dart
Usage guide	webview_flutter loads assets/usage.html with JavaScript enabled	usage/views/usage_view.dart
Theme	Light theme only	core/theme/

Android. applicationId and namespace com.suguna.suguna_assignment_app; label "Zeal"; Java/Kotlin target 17; min/target/compile SDK follow Flutter defaults. Only the INTERNET permission is declared, and usesCleartextTraffic="true" allows plain-HTTP traffic. The release build type signs with the "release" config from key.properties. No app-restrictions schema (APP_RESTRICTIONS meta-data) is declared in the manifest; check with the MDM team how enrollmentNumber is configured on their console.

Native plugins (Android). connectivity_plus, managed_configurations, path_provider, url_launcher, webview_flutter.